

# AI Marketing Content Platform - Project Scope and Final Delivery Plan

Prepared for: SRH University AI Content Creator / AI Marketing Content OS

Date: May 6, 2026

Current status: Working local product with Next.js frontend, FastAPI backend, OpenAI generation, Supabase-ready database design, RAG ingestion, prompt framework, campaigns, repurposing, and feedback loop foundations.

## 1. Executive Summary

This project has evolved from a basic AI content generator into a multi-feature AI marketing content workspace. The current system can generate marketing content, use SRH knowledge-base files, support a modern Next.js interface, connect to a Python API backend, ingest knowledge sources for RAG, manage brand profile data, create campaigns, repurpose content, capture feedback, and evaluate uniqueness against generic model outputs.

The strategic product direction is an AI marketing content operating system for small and medium-sized organizations. The business value is time savings, better brand consistency, safer use of AI-generated content, and reusable knowledge memory across marketing workflows.

The project is not yet fully production complete. The main remaining work is hardening authentication, completing Supabase persistence across all workflows, migrating legacy Python prompt flows where appropriate, improving tenant isolation in the full frontend experience, adding automated tests around the full API and frontend flows, adding monitoring, and preparing deployment infrastructure.

## 2. Product Vision

The product should help marketing teams create high-quality, brand-consistent content faster by combining:

- AI generation
- brand memory
- client/project knowledge bases
- RAG retrieval
- campaign planning
- content repurposing
- feedback loops
- saved output history
- uniqueness evidence
- multi-tenant client/project organization

The final product should feel like a professional SaaS workspace, not a prototype. A user should be able to select a client and project, upload knowledge, define a brand profile, generate content, refine it, save it, repurpose it, schedule it, and review past assets.

## 3. Current Architecture

The current architecture is split into frontend, API proxy, Python backend, and Supabase-ready persistence.

Next.js frontend

- > Next.js API routes
- > FastAPI Python backend
- > OpenAI API
- > Supabase Postgres and pgvector

## **Current frontend**

The frontend is built with:

- Next.js App Router
- TypeScript
- Tailwind CSS
- shadcn-style UI components
- Vercel AI SDK package installed
- API proxy routes under 'frontend/app/api'

Main frontend pages include:

- Dashboard
- Generator
- Clients/projects
- Brand profile
- Knowledge base
- Campaign generator
- Content repurposer
- Content calendar
- Generated assets library
- Settings

## **Current backend**

The backend is built with:

- FastAPI
- Python service modules under 'src'
- OpenAI integration through 'src/llm\_integration.py'
- RAG ingestion through 'src/rag\_ingestion.py'
- Supabase admin client through 'src/supabase\_client.py'
- campaign and repurposing services
- security helpers for rate limiting, API key checks, request size checks, and upload validation

## **Current database direction**

Supabase migrations exist for:

- organizations
- organization members
- clients
- projects
- brand profiles
- uploaded documents
- document chunks
- document embeddings using pgvector

- content briefs
- generated outputs
- campaigns
- campaign items
- content calendar items
- feedback events
- output versions

This is the right foundation for SaaS multi-tenant usage, but the app still needs more complete end-to-end use of these tables.

## 4. Implemented Features

### 4.1 Content Generator

#### What it does

The generator lets a user choose a content type, topic, audience, language, tone, length, knowledge-base strategy, and optional files. It sends the request to the backend and returns generated content plus source metadata.

#### How it works

1. User fills the generator form in the Next.js UI.
2. 'frontend/app/api/generate/route.ts' receives the request.
3. The generation router decides whether to use:
  - legacy Python generation, or
  - TypeScript prompt framework generation when enabled.
4. The Python backend can load knowledge-base context from local Markdown files.
5. The OpenAI wrapper sends the prompt to the configured model.
6. The frontend displays the result, metadata, and feedback controls.

#### Current status

Working locally. The legacy Python route is still the default safe path. The TypeScript prompt framework can be enabled behind feature flags.

#### Missing for production

- Persist every generated output reliably to Supabase.
- Add authenticated user IDs to generated output records.
- Add project/client selectors that are backed by live Supabase data.
- Add robust output status workflow: draft, approved, scheduled, published, archived.
- Add automated tests for full generation request and response behavior.

### 4.2 TypeScript Prompt Engineering Framework

#### What it does

The TypeScript prompt framework provides reusable, versioned prompt templates and orchestration for better, less generic output.

It includes:

- prompt templates
- prompt builder

- anti-generic constraints
- multi-step orchestration
- A/B prompt variants
- uniqueness evaluation
- prompt metadata
- prompt preview and evaluation API routes

## How it works

The framework builds prompts from:

- user input
- brand profile
- retrieved RAG documents
- campaign context
- output format requirements
- distinctiveness constraints

It can run a multi-step pipeline:

1. Strategy generation
2. Messaging angle selection
3. Draft generation
4. Refinement
5. Evaluation

## Current status

Implemented under:

- 'frontend/lib/ai'
- 'frontend/lib/generation'
- 'frontend/app/api/prompts/preview'
- 'frontend/app/api/prompts/evaluate'

It is wired into real generation behind feature flags:

```
ENABLE_TS_PROMPT_FRAMEWORK=true
TS_PROMPT_FRAMEWORK_MODE=social_only
TS_PROMPT_AB_TEST=false
```

Supported first:

- social / social\_post
- newsletter / email
- ad / ad\_copy

## Missing for production

- Persist prompt runs in a dedicated 'prompt\_runs' table.
- Store prompt inputs, final prompt, model, output, evaluation score, and selected variant.
- Connect A/B selection to real feedback and approval metrics.
- Add model-level cost and latency tracking per prompt run.
- Add human review tools for prompt quality.
- Migrate or unify Python prompt templates only after parity tests prove quality is better.

## 4.3 Uniqueness Evaluation

### What it does

The system can demonstrate that its output is different from a generic ChatGPT response.

It compares:

- generic baseline output
- system output

Metrics include:

- cosine similarity over lexical vectors
- lexical diversity
- sentence variation
- distinct term ratio
- generic phrase count
- uniqueness score

### How it works

1. Generate or provide a baseline generic response.
2. Generate the platform response.
3. Run the uniqueness evaluator.
4. Produce a report with score, similarity, and recommendation.

### Current status

Implemented in:

- 'frontend/lib/ai/evaluation/uniqueness.ts'
- 'docs/uniqueness\_evidence\_demo.md'
- 'examples/uniqueness\_comparison.md'

### Missing for production

- Use embedding-based semantic similarity with the same embedding model used by RAG.
- Store evaluation reports in Supabase.
- Show uniqueness evidence in the UI.
- Track uniqueness scores over time by client, project, content type, and prompt version.

## 4.4 Brand Profile

### What it does

Brand profiles define how the AI should speak for a client or project.

Brand profile fields include:

- positioning
- voice
- tone guidelines
- audience summary
- approved terms
- banned terms

- compliance notes
- brand values

### **How it works**

The frontend lets users configure brand profile fields. The backend can retrieve and assemble a brand block for prompt injection. The prompt framework can also use brand context when building TypeScript prompts.

### **Current status**

Brand profile UI and backend routes exist. Supabase schema supports brand profiles. Prompt integration exists.

### **Missing for production**

- Full authentication-aware editing permissions.
- Brand profile version history.
- Approval workflow for brand changes.
- Brand consistency scoring shown in the frontend for generated outputs.
- Required brand profile selection per client/project.

## **4.5 Knowledge Base and RAG Ingestion**

### **What it does**

The platform allows users to upload or paste knowledge sources so AI generation can use client-specific context.

Supported source types include:

- PDF
- DOCX
- TXT
- Markdown
- manually pasted website text
- manually pasted marketing copy

### **How it works**

The RAG pipeline:

1. Receives uploaded or pasted content.
2. Extracts text.
3. Cleans text.
4. Chunks content.
5. Generates embeddings.
6. Stores documents, chunks, metadata, and embeddings in Supabase.
7. Retrieves relevant chunks by client, project, content type, source kind, language, and channel.

### **Current status**

Implemented in:

- 'src/rag\_ingestion.py'
- knowledge ingestion API routes
- Supabase pgvector migration

- frontend knowledge-base page

### **Missing for production**

- Authenticated upload ownership.
- Virus scanning for files.
- Background job queue for large ingestion.
- Upload progress tracking.
- Retryable embedding jobs.
- Deduplication UI.
- Document deletion and re-indexing workflow.
- Better chunk quality evaluation.
- Search UI that explains why a chunk was retrieved.

## **4.6 Campaign Generator**

### **What it does**

The campaign generator creates multi-channel campaign concepts and assets.

It can produce:

- campaign concept
- core message
- audience angle
- channel strategy
- content ideas
- CTA suggestions
- calendar draft
- channel-specific assets

### **How it works**

The Python campaign service:

1. Validates campaign request.
2. Loads knowledge-base context.
3. Retrieves brand profile if provided.
4. Generates a campaign concept as JSON.
5. Generates per-channel assets.
6. Saves campaign, outputs, campaign items, and calendar entries in Supabase when configured.

### **Current status**

Campaign generation is implemented and should remain Python-backed for now.

### **Missing for production**

- More robust JSON repair and validation.
- Campaign editing UI.
- Campaign approval workflow.
- Calendar drag-and-drop scheduling.
- Campaign performance metadata.
- Better regeneration controls per asset.
- Prompt framework migration only after parity testing.

## 4.7 Content Repurposer

### What it does

The repurposer transforms existing content into platform-specific formats while preserving meaning.

Supported formats include:

- LinkedIn
- Instagram
- email
- blog summary
- ad copy
- landing page
- video script

### How it works

The Python repurposing service uses a two-stage process:

1. Extract the structure and meaning of the source content.
2. Generate adapted outputs for target formats.

It also saves lineage using 'parent\_output\_id', so repurposed outputs can be traced back to their source.

### Current status

Implemented in Python and connected to the frontend.

### Missing for production

- Do not migrate this yet until the TypeScript framework has equivalent extraction and lineage handling.
- Add source-to-output visual lineage in the UI.
- Add stronger factual preservation checks.
- Add per-format approval and editing workflow.
- Add batch repurposing for campaign assets.

## 4.8 Content Calendar

### What it does

The content calendar is intended to organize scheduled content across campaigns, channels, and projects.

### Current status

The schema supports calendar items. The frontend page exists as part of the workspace. Campaign generation can produce calendar draft data.

### Missing for production

- Full calendar CRUD.
- Drag-and-drop scheduling.
- Channel filters.
- Approval state filters.

- Publish/schedule integration with external platforms.
- Conflict warnings.
- Export to CSV or calendar systems.

## 4.9 Generated Assets Library

### What it does

The generated assets library is intended to store and review all generated outputs across projects.

### Current status

Supabase schema supports 'generated\_outputs'. The frontend has an assets page. Some flows can persist generated outputs when Supabase is configured.

### Missing for production

- Fully database-backed asset library.
- Search and filters by client, project, campaign, channel, language, status, and date.
- Version history UI.
- Asset duplication and derivation tracking.
- Bulk export.
- Final approval workflow.

## 4.10 Feedback Loop

### What it does

Users can approve generated content, mark it as needing revision, and provide comments.

### How it works

The current backend can store feedback as JSONL. The schema also includes feedback events and output versions for future Supabase-backed persistence.

### Current status

Basic feedback flow exists.

### Missing for production

- Move feedback persistence fully to Supabase.
- Link feedback to generated output IDs.
- Capture edit history.
- Use feedback to improve prompt variant selection.
- Add dashboard metrics for approval rate and revision rate.

## 4.11 Security and Hardening

### What is implemented

The backend now includes:

- optional backend API key validation

- CORS configuration
- request size validation
- upload size and MIME validation
- simple in-memory rate limiting
- sanitized server errors
- LLM prompt and output token guardrails

### **Missing for production**

- Real user authentication.
- Role-based authorization.
- Organization membership enforcement in frontend and API.
- Redis or database-backed rate limiting.
- Security audit for Supabase RLS policies.
- File virus scanning.
- Structured audit logs.
- Secret rotation policy.

## **5. What Still Needs To Be Migrated**

### **5.1 Legacy Python prompt templates**

Current status:

- Python prompt templates still power the default generator, campaign generator, and repurposer.

Migration recommendation:

- Keep Python prompts until the TypeScript prompt framework has proven better quality through tests and comparison mode.
- Migrate one use case at a time.
- Start with social and email generation.
- Leave campaigns and repurposing until lineage, JSON schema validation, and regeneration are safely supported.

### **5.2 Repurposing pipeline**

Current status:

- Repurposing is Python-backed and should remain that way for now.

Required before migration:

- TypeScript extraction schema.
- Source intelligence object parity.
- Lineage persistence with 'parent\_output\_id'.
- Regeneration from stored extraction.
- Tests comparing Python and TypeScript outputs.

### **5.3 Campaign generation**

Current status:

- Campaign generation is Python-backed.

Required before migration:

- TypeScript campaign schema validation.
- JSON repair and retry strategy.
- Campaign concept parity tests.
- Asset persistence parity.
- Calendar item persistence parity.

## 5.4 Feedback persistence

Current status:

- File-backed JSONL feedback exists.
- Supabase schema supports feedback events.

Required migration:

- Save feedback directly to Supabase.
- Link feedback to generated output records.
- Add feedback analytics.
- Use feedback in prompt variant selection.

## 5.5 Asset history

Current status:

- Some frontend history is browser-session based.
- Supabase has the target schema.

Required migration:

- Use 'generated\_outputs' as the source of truth.
- Add output version creation on every edit/regeneration.
- Add asset search and filtering.

## 6. Production Completion Checklist

### Must-have before real client production

- Real authentication.
- Supabase RLS tested end to end.
- Client/project selector backed by database.
- All generated outputs persisted to Supabase.
- Feedback events persisted to Supabase.
- File upload safety, including virus scanning.
- Background jobs for ingestion.
- Monitoring for LLM cost, latency, failures, and token usage.
- Automated backend and frontend tests.
- Deployment documentation.
- Environment variable validation at startup.
- Backup and recovery plan.

### Should-have for best final delivery

- Prompt run observability table.
- Uniqueness report UI.

- Brand consistency scoring UI.
- Campaign editing and approval workflow.
- Content calendar scheduling workflow.
- Asset library search and filters.
- Export options for Markdown, DOCX, PDF, and CSV.
- Team roles: owner, admin, editor, viewer.
- Analytics dashboard.

### **Later SaaS enhancements**

- Billing and plan limits.
- Organization onboarding.
- Multi-client workspace templates.
- Integration with CMS, email platforms, and social schedulers.
- Human approval queues.
- Prompt experimentation dashboard.
- Usage-based cost reporting.

## **7. Recommended Final Delivery Roadmap**

### **Phase 1 - Stabilize the current product**

Goal: Make the existing local product reliable and demo-ready.

Tasks:

- Confirm backend and frontend startup from README.
- Confirm generation, campaign, repurpose, knowledge ingestion, and brand profile flows.
- Fix known UI warnings where useful.
- Add smoke tests for main API routes.
- Add seed data for a clean demo workspace.

### **Phase 2 - Complete Supabase persistence**

Goal: Make Supabase the source of truth.

Tasks:

- Persist all generated outputs.
- Persist feedback events.
- Persist output versions.
- Load assets/history from Supabase.
- Load clients/projects from Supabase.
- Enforce organization/client/project isolation.

### **Phase 3 - Production security**

Goal: Make the product safe for real client data.

Tasks:

- Add authentication.
- Enforce role-based access.
- Validate RLS policies.

- Add production rate limiting.
- Add file scanning.
- Add audit logging.
- Add secret validation and rotation guidance.

## Phase 4 - Quality and uniqueness

Goal: Prove that the platform produces better content than generic LLM usage.

Tasks:

- Add UI for uniqueness comparison.
- Store uniqueness reports.
- Add brand consistency scoring.
- Track approval and revision rates.
- Use feedback to choose prompt variants.

## Phase 5 - Workflow completion

Goal: Turn the app into a complete marketing workspace.

Tasks:

- Finish generated assets library.
- Finish content calendar.
- Add campaign editing.
- Add repurposing lineage UI.
- Add export and approval workflows.

## 8. Final Delivery Definition

The project reaches maximum final delivery when a real client can:

1. Log in securely.
2. Select their organization, client, and project.
3. Upload knowledge sources.
4. Define or edit a brand profile.
5. Generate content using brand and RAG context.
6. Compare output against generic baseline.
7. Review uniqueness and brand consistency evidence.
8. Save, edit, approve, and version outputs.
9. Build campaigns and campaign assets.
10. Repurpose content across channels.
11. Schedule assets in a content calendar.
12. Review history, feedback, and analytics.
13. Trust that data is isolated between clients.

## 9. Current Risk Summary

| Risk                | Level | Reason                                                | Mitigation                                            |
|---------------------|-------|-------------------------------------------------------|-------------------------------------------------------|
| Incomplete auth     | High  | Real users and roles are not fully implemented        | Add Supabase Auth or equivalent before production     |
| Partial persistence | High  | Some flows still rely on session/local/file state     | Move outputs, feedback, and history fully to Supabase |
| File safety         | High  | Uploads need virus scanning and background processing | Add scanning                                          |

and job queue |  
| Prompt migration gaps | Medium | Campaign and repurpose are still Python-backed | Keep behind rollback path until parity is proven |  
| Cost visibility | Medium | LLM usage is controlled but not fully reported | Add token and cost tracking |  
| RLS validation | Medium | Schema has RLS, but end-to-end auth testing remains | Add RLS test suite |  
| UI completeness | Medium | Some pages are scaffolded or partially connected | Complete database-backed workflows |

## 10. Conclusion

The project has a strong foundation and is much more advanced than a simple AI generator. It already includes the core architecture needed for a scalable AI marketing content product: modern frontend, backend services, prompt framework, RAG pipeline, Supabase schema, campaigns, repurposing, feedback, and uniqueness evidence.

The next stage should focus less on adding new features and more on completing the production path: authentication, persistence, tenant isolation, observability, tests, and workflow polish. Once those pieces are complete, the product can be positioned as a serious AI marketing operating system for client-facing use.